

UC LINA 600 LINEAR ALGEBRA FOR GRAPHICS · MODULE 10 · LECTURE NOTES

Eigenvalues and Eigenvectors

The directions a transform does not turn.

MODULE OUTCOMES

Define eigenvectors and eigenvalues geometrically.

Compute them for small matrices.

Explain diagonalisation and what it is for.

Explain the symmetric case and why it is special.

Recognise eigen-problems in the program ahead.

1 The definition

AN EIGENVECTOR IS A DIRECTION THE TRANSFORM ONLY STRETCHES — IT COMES OUT ALONG THE SAME LINE

Most vectors get turned as well as stretched by a transform; eigenvectors do not — they come out as a scalar multiple of themselves, still on the same line through the origin — and that scalar is the eigenvalue.

$Av = \lambda v$ — which says 'on this particular direction, the transform acts like simple scaling', and that is the entire content of the definition.

So a rotation in the plane has no real eigenvectors at all — everything turns — while a rotation in three dimensions has exactly one: its axis, with eigenvalue 1, since the axis is precisely the direction that does not move.

Which is a good first example to hold onto, because 'the axis of rotation is the eigenvector with eigenvalue 1' makes the definition concrete and is directly useful (Module 11's axis-angle representation is this fact, applied).

2 Computing them

$Av = \lambda v$ means $(A - \lambda I)v = 0$ for some nonzero v , which requires $A - \lambda I$ to be SINGULAR, so

$$\det(A - \lambda I) = 0$$

-- the characteristic polynomial (Module 09 gives the determinant condition). Solve it for λ , then solve $(A - \lambda I)v = 0$ for each root.

EXAMPLE $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$

$$\begin{aligned} \det(\begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix}) &= (2-\lambda)^2 - 1 \\ &= \lambda^2 - 4\lambda + 3 \\ &= (\lambda-1)(\lambda-3) \end{aligned}$$

$$\lambda = 1: v = (1, -1)$$

$$\lambda = 3: v = (1, 1)$$

and the two are perpendicular, because A is symmetric (section 4).

BUT NOT AT SCALE: root-finding on a degree- n polynomial is numerically catastrophic -- the roots are wildly sensitive to the coefficients. Real implementations use iterative methods (QR iteration, power iteration). Use a library, and know why you are using it.

The characteristic polynomial is for understanding and for 2×2 examples — real implementations iterate, because polynomial root-finding is numerically catastrophic at any size (Wilkinson's famous example has roots that move enormously under tiny coefficient perturbations). This is a case where the textbook method and the production method are entirely different algorithms, and knowing that is part of knowing the subject.

3 Diagonalisation

IN ITS EIGENVECTOR BASIS A TRANSFORM IS DIAGONAL — WHICH IS CHANGE OF BASIS DOING ITS JOB

$A = PDP^{-1}$, where P 's columns are the eigenvectors and D is the diagonal matrix of eigenvalues — which reads as: change into the eigenbasis, scale each axis independently, change back (Module 04's change of basis, used for its intended purpose).

And then $A^n = PD^nP^{-1}$, where D^n is simply each diagonal entry raised to the n — which turns repeated application of a matrix into exponentiation of a few numbers, and is the reason diagonalisation is worth doing at all.

Which is why eigenvectors matter for anything iterated: the stability of a simulation stepped thousands of times, the convergence rate of an iterative solver, the long-run behaviour of a Markov chain (CSCE 658, CSCE 670's PageRank).

So the question 'does this simulation blow up' becomes 'is the largest eigenvalue greater than 1 in magnitude' — which is CSCE 649's stability analysis and is the most practically important use of this module within this track: it is the difference between an integrator that works and one that explodes after four hundred frames.

4 The symmetric case

- **Real eigenvalues, always — no complex numbers appear**, which is emphatically not true for general matrices (a plane rotation has complex eigenvalues).
- **And mutually orthogonal eigenvectors — so P can be taken orthonormal and $P^{-1} = P^T$** (Module 04 §2), which makes the diagonalisation free to invert and numerically stable.
- **Which together are the spectral theorem, and it is why the symmetric case is the one every application quietly arranges to be in** rather than a lucky coincidence.
- **Inertia tensors are symmetric by construction — so the principal axes of a rigid body are orthogonal, which is why rigid body dynamics is tractable at all** (CSCE 649: in the principal frame the inertia tensor is diagonal and the equations decouple).
- **And covariance matrices are symmetric — so principal component analysis is exactly an eigendecomposition**, with the eigenvectors giving the directions of greatest variance and the eigenvalues giving how much. **This is the single most common use of this module outside graphics** (CSCE 633, CSCE 753, CSCE 676).

AND WHERE THIS APPEARS IN THE PROGRAM

CSCE 649: inertia tensors, modal analysis of deformable bodies, and the stability of numerical integrators — all eigenvalue problems, and the last one decides whether your simulation explodes or settles.

CSCE 753 and CSCE 633: principal components and spectral clustering — the covariance matrix's **eigenvectors are the directions of greatest variance**, and dimensionality reduction is keeping the largest few.

CSCE 645: mesh smoothing, parameterisation, and spectral mesh processing use the Laplacian's spectrum, which is this same machinery applied to a graph rather than to a geometric transform.

And Module 11's quaternions encode the axis-angle representation of a rotation — where the axis is **precisely the eigenvector with eigenvalue 1** (§1), so this module is that one's prerequisite in a direct way.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
3Blue1Brown, chapters 14 and 15 (free)	§§1 and 3 — eigenvectors geometrically, and the change-of-basis reading of diagonalisation.
MIT 18.06, the eigenvalue lectures (free video)	§§2 to 4 — including the spectral theorem and the applications to difference equations.
Trefethen & Bau, lectures 24 to 29	§2's warning — how eigenvalues are actually computed, and why not by the polynomial. Library copy.
Shlens — A Tutorial on Principal Component Analysis (free)	§4's last point — PCA derived as an eigendecomposition, clearly, and free.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Find the eigenvectors** of five 2×2 matrices by hand.
- 2 **Show a plane rotation has no real eigenvectors.**
- 3 **Find the axis of a 3D rotation** as an eigenvector.
- 4 **Verify $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ numerically** for each.
- 5 **Diagonalise a matrix** and verify $\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1}$.

- 6 **Compute A^{100}** two ways and compare the cost.
- 7 **Find the largest eigenvalue** of a simple integrator and predict stability.
- 8 **Then run the simulation** and confirm the prediction.
- 9 **Verify a symmetric matrix** has real eigenvalues and orthogonal eigenvectors.
- 10 **Compute the principal components** of a small point cloud.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define an eigenvector geometrically, then algebraically.
- 2 Why does a plane rotation have none, and a 3D rotation one?
- 3 How are they computed by hand, and why not at scale?
- 4 State the diagonalisation and read it as three steps.
- 5 Why does it make powers cheap?
- 6 What question does the largest eigenvalue answer?
- 7 What two properties do symmetric matrices have?
- 8 What is that theorem called?
- 9 Why are inertia tensors and covariance matrices convenient?
- 10 Name four places eigenvectors appear in the program.